



CHANDIGARH
UNIVERSITY
Discover. Learn. Empower.

Project Report On
AlgoVisualizer Report

SUBMITTED BY:

Name: Divyama Singh

UID: 25MCA20300

CLASS: MCA

SECTION: 25MCA4A

SUBMITTED TO:

Prof. Gurpreet Dhiman

Algorithm Visualizer

1. Case Study Title with Analysis and Critical Thinking

Title:

Design and Implementation of a Web-Based Algorithm Visualizer Integrating C++ Execution and Interactive UI

Analysis:

In traditional learning environments, algorithms are often taught using static examples, textbook explanations, and dry code representations. This approach makes it difficult for students to fully understand the internal working and dynamic behavior of algorithms. Concepts such as sorting, graph traversal, and tree operations involve multiple intermediate steps, which are not easily grasped without a visual or interactive representation. As a result, learners often memorize algorithms rather than truly understanding them.

This problem becomes more significant in topics where step-by-step execution plays a critical role, such as comparing elements in sorting algorithms, exploring nodes in graph traversal, or maintaining structure in trees. Without visualization, these processes remain abstract and difficult to interpret.

This project addresses these challenges by combining three key aspects into a single integrated platform:

- Algorithm implementation (C++) to ensure efficient and real-world execution
- Visual representation (JavaScript animations) to demonstrate step-by-step operations
- Performance understanding (time & space complexity) to analyze efficiency

Unlike basic visualizers that rely only on front-end simulation, this system integrates actual C++ implementations with a web interface through a backend layer. This makes the project more aligned with real-world system design, where multiple technologies interact to deliver a complete solution.

Additionally, the project promotes active learning by allowing users to observe how algorithms behave dynamically rather than passively reading code. This enhances conceptual clarity, improves retention, and encourages deeper engagement with the subject.

Critical Thinking:

While designing and implementing this system, several technical and conceptual challenges were identified and addressed. One of the primary challenges was bridging the gap between low-level C++ execution and high-level web-based user interfaces. Since C++ cannot directly interact with browser-based technologies, a backend layer had to be introduced to facilitate communication.

Another important challenge was ensuring synchronization between algorithm execution and visualization. The system needed to accurately reflect each step of the algorithm through animations, which required careful control of timing and sequencing using asynchronous programming techniques.

Designing a modular and scalable architecture was also a key consideration. The project was structured in such a way that each algorithm or data structure operates independently, making it easier to add new features or extend functionality in the future without affecting existing components.

Furthermore, the project required balancing visual clarity with performance. Too many animations could slow down execution, while too little detail could reduce understanding. Therefore, an optimal level of visualization was maintained to ensure both efficiency and clarity.

Overall, the project applies critical thinking by transforming abstract algorithmic logic into interactive visual flows, improving both usability and educational value. It not only demonstrates technical implementation skills but also reflects a problem-solving approach to enhancing the learning experience in the field of Design and Analysis of Algorithms.

2. Project Design and Implementation / Technical Implementation (Tools)

System Architecture:

The project follows a **three-layer architecture**:

1. Frontend Layer

- Technologies: HTML, CSS, JavaScript
- Responsibilities:
 - User interaction
 - Display of code, complexity, and visualization
 - Dynamic UI updates

2. Backend Layer

- Technology: Node.js (server.js)
- Responsibilities:
 - Acts as a bridge between frontend and C++
 - Executes C++ programs using child processes
 - Sends output back to frontend

3. Algorithm Layer

- Technology: C++
- Responsibilities:
 - Implementation of algorithms (sorting, graphs, etc.)
 - Efficient execution logic

Tools and Technologies Used:

C++

- Used for implementing algorithms such as sorting and graph traversal
- Provides high performance and efficiency

HTML

- Structures the interface (panels, buttons, sections)

CSS

- Enhances visual design and layout

JavaScript

- Handles logic for visualization
- Controls animations using asynchronous functions (sleep/delay)

Node.js

- Connects frontend with C++ using backend APIs
- Uses `child_process.exec()` to run compiled C++ programs

Highlight.js

- Provides syntax highlighting for C++ code display

Working Mechanism:

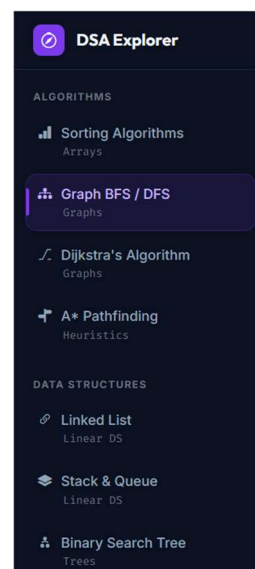
1. User selects an algorithm (e.g., sorting, graph)
 2. JavaScript updates UI dynamically using topic-based rendering
 3. Visualization starts using animation functions
 4. Backend (Node.js) executes C++ logic when required
 5. Output is returned and displayed
 6. UI shows:
 - Code
 - Complexity
 - Visual steps
-

3. Output Screenshots

Description of Outputs:

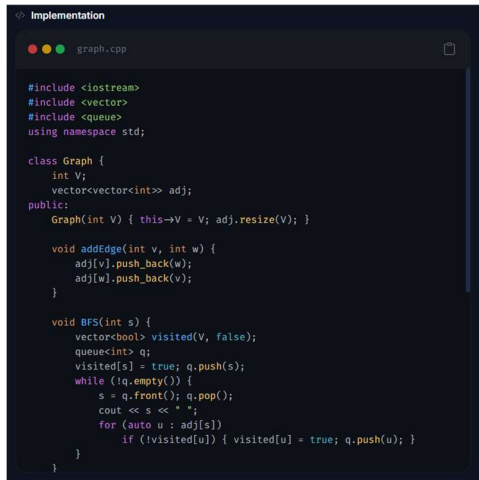
1. Main Dashboard

- Sidebar with topics:
 - Sorting
 - Graph
 - Dijkstra Algorithm
 - A* Pathfinding
 - Linked List
 - Stack & Queue
 - Binary Search Tree



2. Code Section

- Displays C++ code
- Syntax highlighting using Highlight.js



```
Implementation
graph.cpp

#include <iostream>
#include <vector>
#include <queue>
using namespace std;

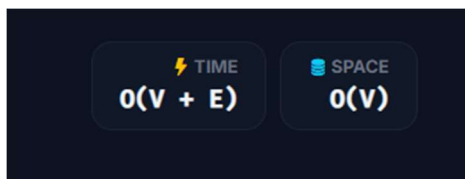
class Graph {
    int V;
    vector<vector<int>>> adj;
public:
    Graph(int V) { this->V = V; adj.resize(V); }

    void addEdge(int v, int w) {
        adj[v].push_back(w);
        adj[w].push_back(v);
    }

    void BFS(int s) {
        vector<bool> visited(V, false);
        queue<int> q;
        visited[s] = true; q.push(s);
        while (!q.empty()) {
            s = q.front(); q.pop();
            cout << s << " ";
            for (auto u : adj[s])
                if (!visited[u]) { visited[u] = true; q.push(u); }
        }
    }
};
```

3. Complexity Section

- Shows time and space complexity
- Helps in comparing algorithms



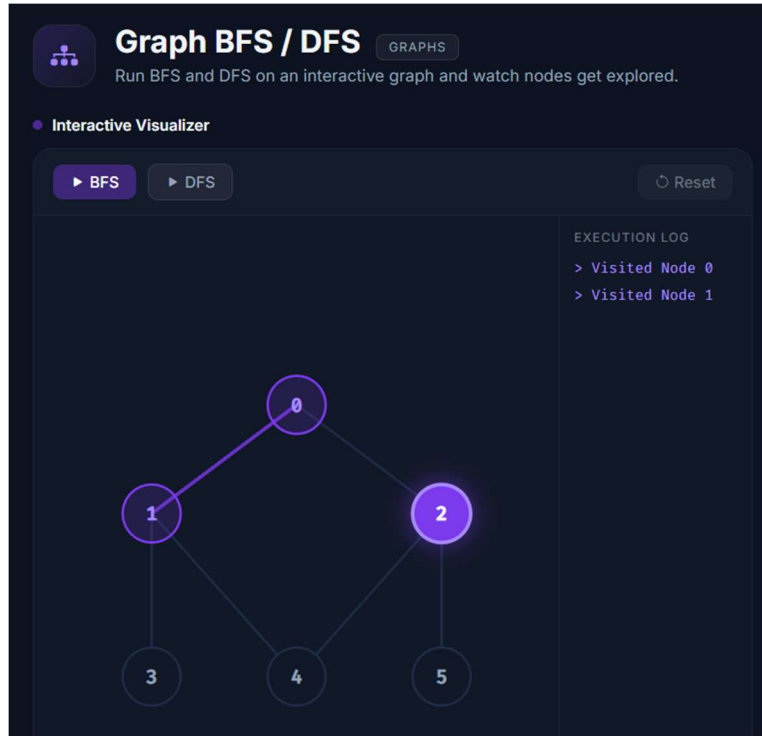
4. Sorting Visualization

- Bars represent array elements
- Swapping and comparisons shown using animation



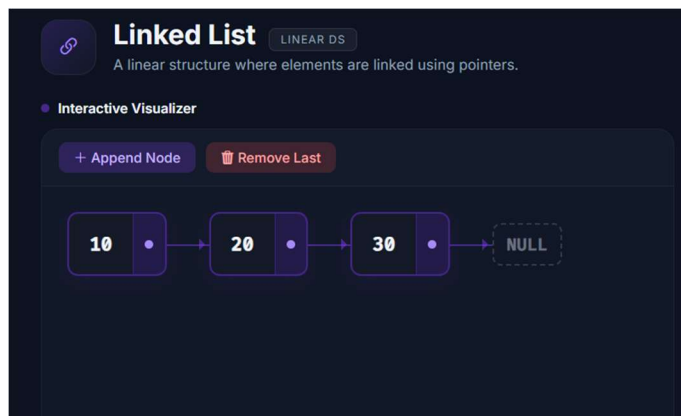
5. Graph Visualization

- Nodes and edges displayed
- BFS/DFS traversal shown step-by-step



6. Data Structure Visualization

- Linked List → Node connections
- Stack → Push/Pop operations
- Queue → Enqueue/Dequeue operations
- BST → Tree structure representation



4. Novelty and Limitations of the Study

Novelty:

1. Hybrid Architecture Integration

This project uniquely combines low-level programming (C++) with high-level web technologies (HTML, CSS, JavaScript) using a Node.js backend. This integration is not commonly seen in basic academic projects and demonstrates a deeper understanding of full-stack system design.

2. Real-Time Algorithm Visualization

Unlike traditional static learning methods, this system provides real-time, step-by-step animation of algorithm execution. Users can visually track operations such as comparisons, swaps, node traversal, and structural changes.

3. Multi-Domain Coverage

The project is not limited to a single category but supports multiple domains including:

- Sorting algorithms
- Graph traversal (BFS/DFS)
- Linear data structures (Stack, Queue, Linked List)
- Tree structures (Binary Search Tree)

This makes it a comprehensive learning platform.

4. Integrated Learning Approach

The platform combines three essential learning components in one interface:

- Source code (C++)
- Time and space complexity
- Visual execution

This integrated approach enhances both conceptual and practical understanding.

5. Modular and Scalable Design

The system is designed in a modular way where each topic has its own visualization logic. New algorithms or data structures can be easily added without affecting existing components.

6. Interactive and User-Friendly Interface

The use of JavaScript allows dynamic UI updates, smooth animations, and responsive design, making the platform intuitive even for beginners.

7. Educational Impact

This project can serve as a teaching aid in classrooms and self-learning platforms. It transforms complex algorithmic logic into visually understandable patterns, improving retention and engagement.

Limitations:

1. Limited Algorithm Coverage

Currently, the project includes only fundamental algorithms and data structures. Advanced algorithms like shortest path (Dijkstra), dynamic programming, or greedy algorithms are not yet implemented.

2. Backend Dependency

The system relies on Node.js to execute C++ programs. This creates a dependency on backend setup, which may limit portability and ease of deployment for non-technical users.

3. Performance Constraints

Visualization involves animations and delays, which can slow down execution for large datasets. The system is more suitable for educational demonstration rather than handling large-scale data.

4. Limited User Input Flexibility

The current implementation provides predefined datasets or limited interaction. Users may not have full control over custom input values, which reduces flexibility.

5. Basic UI/UX Design

Although functional, the UI is relatively simple. It lacks advanced features such as drag-and-drop interaction, real-time speed control sliders, or modern UI frameworks.

6. Synchronization Complexity

Maintaining synchronization between backend execution (C++) and frontend visualization (JavaScript) is complex and may lead to slight inconsistencies if not handled carefully.

7. Platform Dependency

The execution of C++ programs may vary depending on the system configuration, compiler setup, and environment, which can affect portability.

8. No Persistent Data Storage

The system does not store user progress or history, which limits its use as a long-term learning platform.

Future Scope:

1. Addition of Advanced Algorithms

Include algorithms like Dijkstra, A*, Floyd-Warshall, and dynamic programming problems for broader coverage.

2. Enhanced User Interaction

Allow users to input custom data, control animation speed, and pause/resume execution.

3. Improved UI/UX

Use modern frameworks like React or Vue.js to create a more interactive and visually appealing interface.

4. Performance Optimization

Optimize animations and execution logic to handle larger datasets efficiently.

5. Web Deployment

Deploy the application on cloud platforms so it can be accessed globally without local setup.

6. Algorithm Comparison Feature

Allow users to compare multiple algorithms side-by-side based on time, space, and execution behavior.

7. Learning Analytics

Track user interactions and provide insights or recommendations for better learning.

Conclusion

This project successfully integrates algorithm logic with interactive visualization using a hybrid architecture of C++, Node.js, and frontend technologies. It enhances learning by making abstract concepts intuitive and engaging, thereby serving as an effective educational tool in the field of Design and Analysis of Algorithms.

References

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.
2. Kleinberg, J., & Tardos, É. (2006). *Algorithm Design*. Pearson Education.
3. Weiss, M. A. (2014). *Data Structures and Algorithm Analysis in C++* (4th ed.). Pearson.
4. Skiena, S. S. (2020). *The Algorithm Design Manual* (3rd ed.). Springer.
5. GeeksforGeeks. (n.d.). *Data Structures and Algorithms Tutorials*. Retrieved from <https://www.geeksforgeeks.org>
6. Visualgo. (n.d.). *Algorithm Visualization Tool*. Retrieved from <https://visualgo.net>
7. W3Schools. (n.d.). *Web Development Tutorials*. Retrieved from <https://www.w3schools.com>
8. Mozilla Developer Network (MDN). (n.d.). *JavaScript Documentation*. Retrieved from <https://developer.mozilla.org>
9. Node.js. (n.d.). *Node.js Documentation*. Retrieved from <https://nodejs.org>

Github Link

<https://github.com/divyamasingh2002/AlgoVisualizer>
